

Bachelor's Thesis

Implementing Floating-Point Support in a Nanopass-Python Compiler

Marius Spill

Examiner: Prof. Dr. Peter Thiemann

Advisers: TO DO

Albert-Ludwigs-University Freiburg

Faculty of Engineering

Department of Computer Science

Chair for Programming Languages

September 15, 2026

Writing period

06.15.2026 – 09.15.2026

Examiner

Prof. Dr. Peter Thiemann

Advisers

TODO

Declaration

I hereby declare that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I hereby also declare that my Thesis has not been prepared for another examination or assignment, either wholly or excerpts thereof.

Place, Date

Signature

Abstract

foo bar

Contents

1 Introduction

1.1 Subchapter 1

The Hook: Why do we care about small/incremental compiler frameworks (like Nanopass)?

1.2 Subchapter 2

The Problem: What happens when a framework or downstream intermediate representation (IR) lacks native floating-point types? (e.g., restriction to purely integer arithmetic limits the computational expressiveness of the language). The Contribution: A brief high-level summary of what you added to the pipeline.

2 Background

The Nanopass Framework: Explain how the host compiler operates (small, discrete passes transforming highly specific AST micro-languages). Floating-Point Representation: Briefly introduce the target representation semantics (e.g., IEEE 754 standards) and how the target machine or runtime expects floats to be structured compared to integers.

3 Design Architecture

AST Extensions: What new node types did you introduce to represent float literals, float primitives, and operators? Pass Pipeline Changes: Walk the reader through the compilation pipeline. Which existing passes had to change to propagate float types, and what new passes did you have to insert to deal with float semantics?

4 Implementation

The Engineering Gritty Details: How did you handle code generation for floating-point operations? Type Typing/Checking: How does your compiler catch type mismatches (e.g., trying to add an integer to a float if implicit casting isn't supported)? Runtime/Backend Integration: If there is a runtime system or specific assembly/IR emitted (like x86 SSE registers or LLVM float primitives), explain how you lowered the AST nodes to it.

4.0.1 Interpreter extension

- parser: extend `astpython` to have `accept floats`
- parser: just add cases for where the python parser returns float is accepted.
- parser: add a case for float variable annotation (e.g `a : float = 3.14`)
- `type_checker` : `extend types.py` to have a `TFloat()` dataclass (just boilerplate code)
- `type_checker` : `hardcoded TInt()` in various places, exchange for either `TInt()` or `TFloat()`

5 Evaluation

Correctness Validation: Instead of standard benchmarks or accuracy percentages, compiler evaluations validate determinism and compliance. Explain your test suite framework. Test Cases: Discuss how you validated corner cases (e.g., parsing precision limits, type coercion, complex floating-point expressions). Performance (Optional): If you optimize passes or look at compilation times, present it here. Otherwise, focus heavily on functional correctness and compilation validation.

6 Conclusion

Short summary of your execution and a couple of bullet points on future work (e.g., supporting double-precision, floating-point optimization passes, or array allocations of floating-point numbers).

